MODELAGEM DE PROCESSOS E NEGÓCIOS

AULA 4

Profª Adriana Bastos da Costa

CONVERSA INICIAL

Neste estudo, estamos discutindo os conceitos relacionados ao mapeamento de processos, que auxiliam na compreensão do funcionamento das empresas e de seus processos.

Anteriormente, reforçamos os conceitos fundamentais de modelagem de processos. Também, discutimos diferentes tipos e técnicas utilizadas no mapeamento de processos, aprofundando-nos no uso do BPM e da UML, além de estudarmos o funcionamento de cada técnica para deixar mais clara a diferença entre elas. Aprofundamos o funcionamento do Business Process Model and Notation (BPMN), compreendendo como a notação está organizada e estudando um exemplo de modelagem de processo utilizando essa ferramenta.

Até o momento, destacamos a importância de mapear os processos de negócio antes de compreender, planejar tecnicamente e construir um software. Vimos que o uso de uma ferramenta não é obrigatório, mas facilita muito todo o processo de modelagem de negócios.

Nesta etapa, vamos abordar a modelagem de negócios utilizando a UML. Exploraremos os principais conceitos envolvidos na modelagem de negócios, além de compreendermos cada um dos diagramas previstos na UML, aprofundando-nos nessa técnica e entendendo as diferenças em relação à modelagem que utiliza a técnica do BPMN.

Esta etapa estará dividida em 5 temas principais, os quais estão relacionados entre si e ajudarão na compreensão dos conceitos propostos. Resumidamente, trataremos dos seguintes assuntos:

- Introdução à UML;
- Diagrama de caso de uso na modelagem de negócios;
- Diagrama de classe na modelagem de dados e processos de negócios;
- Diagrama de atividade e a modelagem de fluxo de trabalhos em processos de negócios;
- Diagrama de sequência e a modelagem de interações entre objetos no contexto de negócios.

TEMA 1 – INTRODUÇÃO À UML

A UML (Unified Modeling Language) é uma linguagem de modelagem amplamente utilizada para representar visualmente sistemas de software, permitindo aos desenvolvedores, analistas e usuários entenderem, comunicarem e documentarem o design e o funcionamento de um sistema.

A UML foi desenvolvida inicialmente na década de 1990 por Grady Booch, James Rumbaugh e Ivar Jacobson, que uniram suas metodologias de modelagem de software para criar uma linguagem unificada. Em 1997, a UML foi adotada como um padrão pela OMG (Object Management Group), uma organização internacional de padronização de TI.

O objetivo principal da UML é fornecer uma linguagem padrão e visual para modelar sistemas de software, permitindo uma comunicação mais clara e eficaz entre todas as partes interessadas, incluindo desenvolvedores, analistas de negócios, gerentes de projeto e clientes.

A UML consiste em vários tipos de diagramas, cada um representando uma perspectiva específica do sistema, e é suportada por uma variedade de ferramentas de modelagem, que permitem aos usuários criar e editar diagramas de UML de forma visual. A notação da UML consiste em símbolos gráficos padronizados, como caixas, setas, linhas e ícones, que representam diferentes elementos e relações nos diagramas.

Essa linguagem de modelagem é amplamente utilizada na indústria de software para modelar sistemas de todos os tamanhos e complexidades, desde pequenas aplicações até sistemas corporativos e de missão crítica. Ela é especialmente útil durante as fases de análise e design de um projeto de software, ajudando a capturar requisitos, definir arquitetura, identificar padrões de design e validar decisões de design.

A UML é uma linguagem poderosa e flexível para modelagem de sistemas de software, que facilita a comunicação, a colaboração e a compreensão entre todas as partes interessadas envolvidas no desenvolvimento de software. Ela fornece uma maneira padronizada e visual de representar conceitos abstratos e complexos, tornando mais fácil para todos os envolvidos entenderem e contribuírem para o sucesso de um projeto de software.

Ao utilizar corretamente a UML, é possível capturar de maneira eficaz os requisitos de negócios por meio de diagramas como o Diagrama de Caso de

Uso. Isso ajuda a identificar os atores (usuários) e suas interações com o sistema, bem como os casos de uso que descrevem as funcionalidades necessárias para atender às necessidades do negócio. Também facilita a análise e validação dos requisitos de negócios, permitindo que as partes interessadas revisem e forneçam feedback sobre os modelos de negócios representados. Isso ajuda a garantir que os requisitos estejam alinhados com as necessidades do negócio e que todos os aspectos relevantes tenham sido considerados.

Através de diagramas como o Diagrama de Componentes e o Diagrama de Implantação, a UML permite visualizar a arquitetura empresarial, identificando os principais componentes de software, suas interações e como são implantados em ambientes de produção. Note-se que cada diagrama da UML é responsável por modelar uma perspectiva do software.

A UML desempenha um papel crucial na modelagem de negócios, ajudando as organizações a entender, comunicar e documentar os aspectos estruturais e comportamentais de seus sistemas de negócios. Ao fornecer uma linguagem visual padronizada e poderosa, a UML ajuda as empresas a capturar requisitos, visualizar arquiteturas, modelar processos e colaborar efetivamente para o sucesso de seus projetos de negócios.

1.1 Diagramas na UML

Os diagramas na UML têm o objetivo de fornecer uma representação visual e compreensível dos diferentes aspectos de um sistema de software. Os diagramas facilitam o entendimento do funcionamento do software, além de apresentarem outros objetivos, tais como:

- Comunicação: os diagramas na UML servem como uma linguagem comum para comunicar ideias, conceitos e informações sobre o sistema entre diferentes partes interessadas, como desenvolvedores, analistas de negócios, clientes e gerentes de projeto. Eles fornecem uma forma visual e intuitiva de transmitir informações complexas de forma clara e concisa.
- Análise e design: os diagramas na UML são ferramentas poderosas para analisar e projetar sistemas de software. Eles permitem que os desenvolvedores visualizem e compreendam a estrutura, comportamento e interações do sistema, facilitando a identificação de requisitos, padrões de design e problemas de projeto.

- Documentação: os diagramas na UML são usados para documentar o design e a arquitetura do sistema, fornecendo uma referência visual para os desenvolvedores e outros membros da equipe. Eles ajudam a capturar e registrar decisões de design, requisitos funcionais e não funcionais, e outras informações importantes sobre o sistema.
- Validação e verificação: os diagramas na UML são usados para validar e verificar o sistema durante as fases de desenvolvimento, teste e manutenção. Eles permitem que os desenvolvedores e testadores revisem e analisem o design do sistema, identificando problemas de usabilidade, inconsistências de requisitos e outras questões que precisam ser resolvidas.
- Visualização de conceitos abstratos: os diagramas na UML são usados para visualizar conceitos abstratos e complexos de forma mais tangível e acessível. Eles transformam ideias e conceitos abstratos em representações visuais claras e tangíveis, permitindo uma compreensão mais fácil e uma comunicação mais eficaz entre as partes interessadas.

Logo, percebemos que os diagramas na UML são ferramentas essenciais para modelar, analisar, projetar, documentar e comunicar sistemas de software de forma eficaz. Eles desempenham um papel fundamental em todo o ciclo de vida do desenvolvimento de software, ajudando as equipes a colaborar, compreender e entregar sistemas de alta qualidade que atendam às necessidades dos clientes e usuários finais.

Vamos analisar os principais diagramas disponíveis na UML:

- Diagrama de caso de uso: descreve as interações entre os atores externos (usuários) e o sistema, identificando os casos de uso (funcionalidades) do sistema e como eles são usados pelos atores.
- Diagrama de classes: representa a estrutura estática do sistema, mostrando as classes do sistema, seus atributos, métodos e os relacionamentos entre elas.
- Diagrama de sequência: mostra a interação entre os objetos do sistema ao longo do tempo, representando a troca de mensagens entre eles em uma sequência temporal.
- Diagrama de comunicação: é equivalente ao diagrama de sequência, mas sem o foco temporal (algumas ferramentas UML geram diagrama de

comunicação a partir do diagrama de sequência). O diagrama de comunicação também pode ser chamado de diagrama de colaboração.

- Diagrama de atividades: modela o fluxo de controle entre as atividades do sistema, mostrando as ações a serem realizadas e as condições para a transição entre elas.
- Diagrama de estado: descreve o comportamento interno de um objeto em resposta a eventos externos, mostrando os diferentes estados em que o objeto pode estar e as transições entre esses estados.
- Diagrama de componentes: representa os componentes físicos do sistema e suas interações, mostrando como os diferentes elementos de software estão agrupados em componentes maiores.
- Diagrama de implantação: descreve a disposição física dos componentes do sistema em hardware de computador, mostrando como eles são implantados e interconectados em um ambiente de execução.

São muitos os diagramas disponibilizados pela UML, e vamos explorar nesta etapa os principais diagramas utilizados pela maioria dos projetos e empresas.

TEMA 2 – DIAGRAMA DE CASO DE USO NA MODELAGEM DE NEGÓCIOS

O Diagrama de Caso de Uso é uma ferramenta fundamental na modelagem de negócios, utilizada para capturar os requisitos funcionais de um sistema sob a perspectiva dos usuários finais.

Ele tem como objetivo identificar os atores (usuários ou sistemas externos) que interagem com o sistema e os casos de uso (funcionalidades) que o sistema oferece para atender às necessidades desses atores.

Dessa forma, os principais elementos em um diagrama de caso de uso são os atores, o próprio caso de uso e os relacionamentos. Vamos analisar cada um desses elementos.

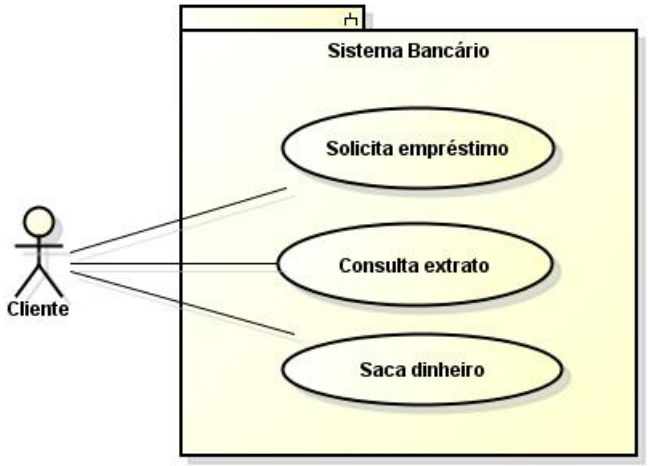
2.1 Ator e caso de uso

O ator representa o papel executado por uma entidade que interage com o sistema em questão. Um ator modela algo fora da fronteira do sistema que precisa trocar informações com o sistema, tais como indivíduos e outros sistemas, ou seja, o ator interage com o sistema, ele não faz parte do sistema.

Um usuário pode executar o papel de vários atores diferentes e um determinado ator pode ser representado por vários usuários. Dessa forma, é preciso esclarecer que o ator não é o mesmo que usuário. Mas, reforçando o conceito:

- Um ator representa um papel exercido por um usuário ao interagir com um determinado caso de uso.
- Usuários podem desempenhar mais de um papel junto ao sistema.
- Em modelagem de software, como a UML é a linguagem utilizada, é preciso conhecer a notação utilizada, ou melhor, os símbolos que representam graficamente os elementos do diagrama de caso de uso.

Um ator é representado por um boneco e um rótulo com o nome do ator. Um ator é um usuário do sistema, que pode ser um usuário humano ou outro sistema computacional.



Os atores podem pertencer a categorias diferentes, e algumas das mais comuns são:

- Usuários: usuários finais, administradores, gerentes, entre outros.
- Aplicações: processos individuais, rotinas automatizadas e sistemas de software externos ao sistema sendo modelado.
- Dispositivos: sensores ou qualquer outro hardware que tenha interação com o sistema sendo modelado.
- Eventos externos: como um controlador de tempo, um escalonador de processos, ou qualquer outro evento que dispare uma funcionalidade do sistema sendo modelado.

Uma boa dica para identificar os atores é analisar as fontes e os destinos das informações a serem processadas, pois estes são atores em potencial, e é importante analisar as áreas da organização que serão afetadas ou utilizarão o sistema, pois estes podem ser atores em potencial.

Algumas perguntas úteis para apoiar na identificação de possíveis atores: que órgãos, empresas ou pessoas irão utilizar o sistema? Que outros sistemas irão se comunicar com o sistema a ser construído? Alguém deve ser informado de alguma ocorrência no sistema? Quem está interessado em um certo requisito funcional do sistema?

- Representação gráfica do caso de uso: seu ícone é uma elipse contendo seu nome. É possível encontrar um diagrama de caso de uso onde o nome do caso de uso é apresentado abaixo da elipse. Um caso de uso define uma função do sistema.

É muito importante iniciar a modelagem do diagrama de casos de uso separando as funcionalidades do sistema, levando em conta os seguintes itens:

- Para que cada funcionalidade ou grupo de funcionalidades seja atendido, deve haver um conjunto ou sequência de ações que tenham um objetivo bem definido e são detalhadas na descrição de caso de uso. Para relembrar este conceito, reveja a Aula 3.
- Durante a execução dessa sequência de ações, o sistema interage com elementos externos a ele, que são os atores.
- É importante se concentrar inicialmente no fluxo principal e depois nos fluxos alternativos e de exceção. Isso garante a completude do escopo do caso de uso. Para relembrar estes conceitos, reveja as regras de descrição de caso de uso, estudadas na Aula 3.
- Descreva O QUE o sistema fará, mas não especifique COMO será feito, pois este não é o objeto desta fase da modelagem do software. O COMO será modelado em outros diagramas que estudaremos nas próximas etapas.

2.2 Relacionamentos

Os relacionamentos mostram a ligação entre os elementos de um diagrama de caso de uso, ou seja, mostram a ligação dos atores com os casos de uso e dos casos de uso entre si.

Os relacionamentos ajudam a compreender o funcionamento do software, mostrando quais atores têm acesso a cada caso de uso e como os casos de uso se relacionam, apresentando a dinâmica de interação entre eles, focando no entendimento do funcionamento dos processos de negócio.

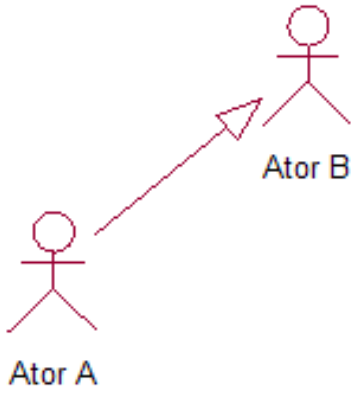
Vamos nos aprofundar nos diferentes tipos de relacionamentos que podem ser encontrados em um diagrama de caso de uso.

2.2.1 Relacionamento entre atores

O relacionamento entre os atores é chamado de Generalização. Este relacionamento é identificado quando temos dois atores semelhantes, mas com um deles realizando algo a mais.

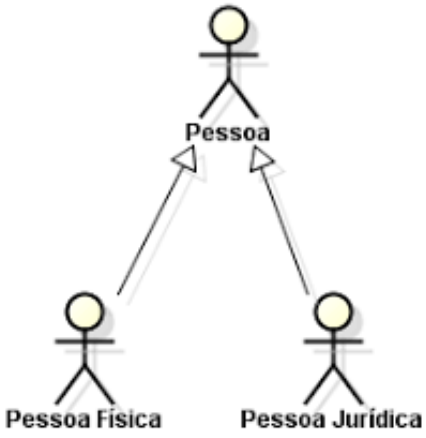
A generalização é usada para identificar funcionalidades comuns entre atores, e pode ser sempre lida como “é-um-tipo-de”. Ela deve ser identificada e validada com cuidado para não modelar relacionamentos desnecessários a um determinado ator, ou mesmo não demonstrar relacionamentos que devem ocorrer para garantir o bom funcionamento do software.

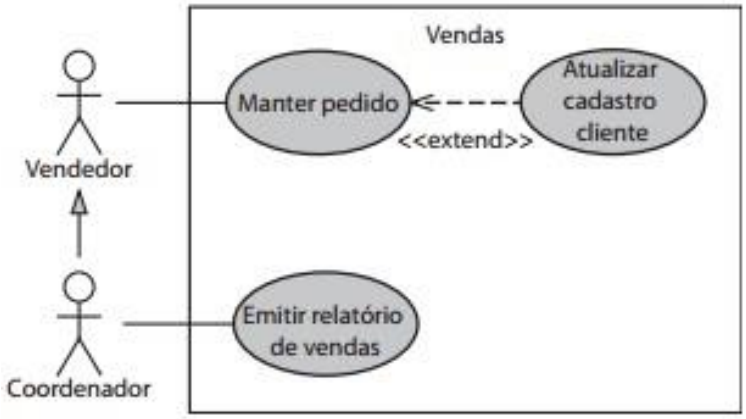
Os relacionamentos precisam representar exatamente as ligações corretas para o software, nem mais e nem menos.



Os casos de uso relacionados com o ator B são também automaticamente relacionados com casos de uso do ator A. E o ator A tem seus próprios casos de uso, aos quais o ator B não tem acesso.

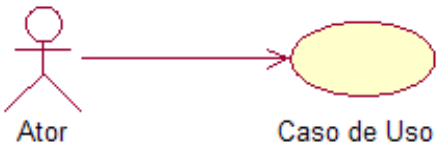
Alguns exemplos de generalizações:





Nos exemplos apresentados, podemos perceber que não importa a quantidade de níveis da hierarquia, o conceito se mantém o mesmo. Ou seja, os usuários dos níveis mais baixos herdam os casos de uso do nível superior, enquanto os casos de uso de nível superior possuem seus próprios casos de uso. Dessa forma é possível reaproveitar casos de uso que podem ser acessados por mais de um ator, permitindo ainda que alguns atores tenham acesso a outros casos de uso, dependendo do uso que terá do sistema em desenvolvimento.

2.2.2 Relacionamento entre um ator e um caso de uso: associação



O relacionamento de associação define uma funcionalidade do sistema do ponto de vista do usuário, mostrando visualmente a quais casos de uso cada ator tem acesso.

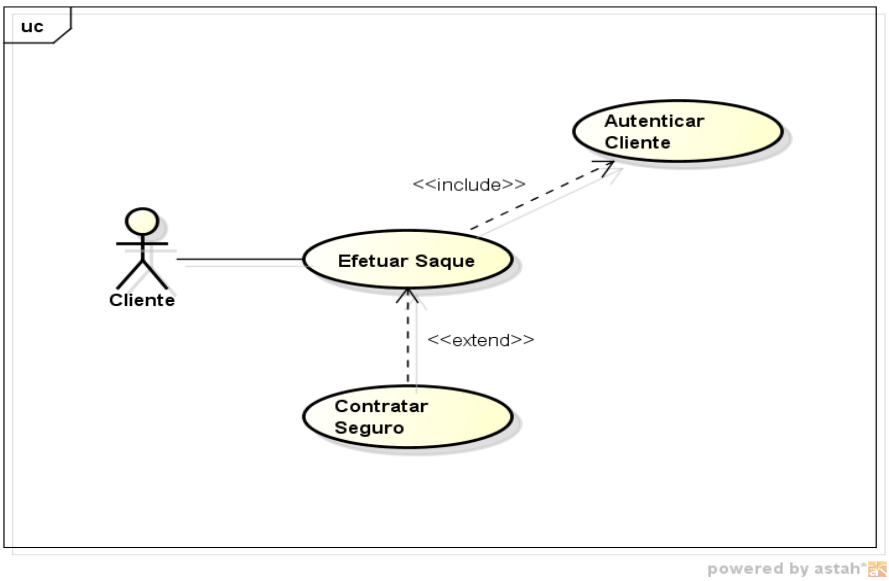
A associação representa a interação do ator com o caso de uso, por meio de envio e recebimento de mensagens. Associações são representadas por uma linha sólida, ligando o ator ao caso de uso. Se o ator inicia um caso de uso, este pode se comunicar com vários atores depois. As associações servem para mostrar quais atores se comunicam com o caso de uso em questão.

A direção do relacionamento de associação também é importante para compreender o funcionamento dos casos de uso. Dessa forma, é preciso compreender os seguintes pontos:

- Cada ator de uma associação tem uma propriedade de navegabilidade, que indica quem inicia a comunicação na interação.
- A direção é representada no diagrama por uma seta: se a seta apontar para um caso de uso — o ator inicia a interação. Se a seta apontar para o ator — o sistema inicia a interação.
- A navegabilidade de duas direções é mostrada por uma linha sem setas. Também é possível usar setas bidirecionais.

2.2.3 Relacionamento entre casos de uso

- Include: um relacionamento include de um caso de uso A para um caso de uso B indica que B é essencial para o comportamento de A. Pode ser dito também que B é parte de A.
- Extend: um relacionamento extend de um caso de uso B para um caso de uso A indica que o caso de uso B pode ser acrescentado para descrever o comportamento de A (não é essencial). Ponto de extensão em um caso de uso é uma indicação de que outros casos de uso poderão ser adicionados a ele. Quando o caso de uso for invocado, ele verificará se suas extensões devem ou não ser invocadas.



Analisando o exemplo acima, em um caso de uso de Saque em um Caixa Eletrônico, percebemos que Autenticar o Cliente é uma ação obrigatória, por isso o caso de uso “Efetuar Saque” faz um “Include” de “Autenticar Cliente”. Já o caso de uso “Contratar um Seguro” é opcional, dependendo do aceite do cliente, por isso ele é chamado com um relacionamento de “Extend”, pois depende de uma ação direta do usuário.

Ou seja, o Diagrama de Caso de Uso é usado para capturar os requisitos funcionais do sistema, descrevendo as interações entre os atores e as funcionalidades que o sistema oferece. Ele é valioso para os analistas de negócios, desenvolvedores e stakeholders entenderem as necessidades e expectativas dos usuários finais, garantindo que o sistema seja projetado e desenvolvido de acordo com esses requisitos.

O Diagrama de Caso de Uso na modelagem de negócios é uma ferramenta essencial para identificar e documentar os requisitos funcionais de um sistema, fornecendo uma representação clara e concisa das interações entre os atores e as funcionalidades oferecidas pelo sistema. Ele desempenha um papel crucial na comunicação, análise e documentação dos requisitos do sistema, garantindo que o sistema atenda às necessidades e expectativas dos usuários finais.

TEMA 3 – DIAGRAMA DE CLASSE NA MODELAGEM DE DADOS E PROCESSOS DE NEGÓCIOS

Um diagrama de classe, segundo a UML, é uma representação visual fundamental na modelagem de dados e processos de negócios, utilizada principalmente na engenharia de software e em metodologias de desenvolvimento orientadas a objetos.

Ele é um tipo de diagrama estático que descreve a estrutura de um sistema mostrando suas classes, atributos, operações (ou métodos) e os relacionamentos entre os objetos. Ou seja, o diagrama de classe mostra todas as classes de um software e os relacionamentos entre elas. Mas para compreender o objetivo do diagrama de classes é preciso entender o que é uma classe.

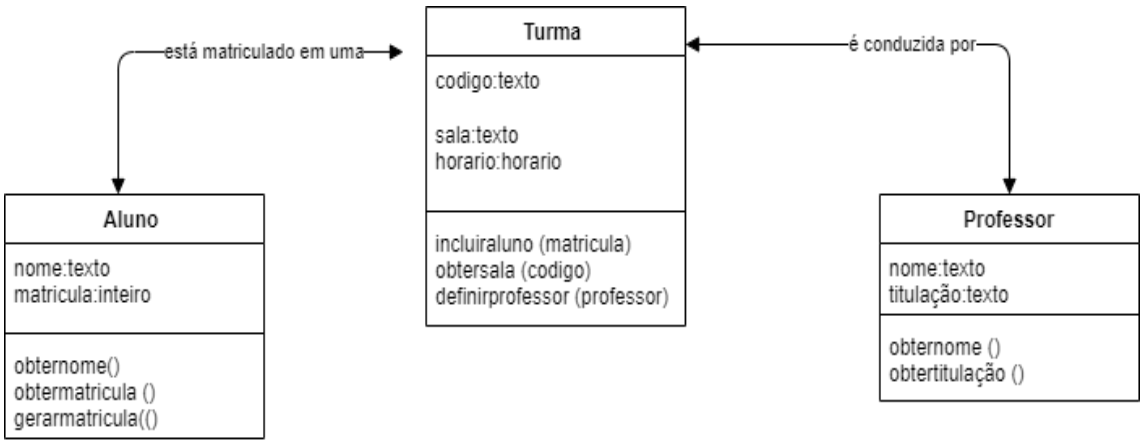
Em UML, uma classe representa um objeto ou um conjunto de objetos que compartilham uma estrutura e comportamento comum. E um objeto é uma representação de um elemento do mundo real. Por exemplo, CLIENTE, ALUNO, PRODUTO, entre outros são exemplos de objetos ou classes.

É por isso mesmo que a modelagem das classes está totalmente relacionada com os conceitos de orientação a objetos. Os principais conceitos da orientação a objetos que podem ser representados no diagrama de classe são:

- Abstração: tem o foco em aspectos relevantes para um determinado propósito, abstraindo os demais elementos que não são importantes para a situação que se está modelando. Por exemplo, para verificar a situação de um ALUNO, se ele está APROVADO ou REPROVADO, é preciso consultar a matrícula dele. Neste contexto não preciso saber o endereço ou o telefone deste aluno. Desta forma, foco em obter a matrícula dele, abstraindo outras informações sobre o ALUNO que não são relevantes para a ação que quero tomar.
- Encapsulamento: envolve a separação dos aspectos externos de um objeto, acessíveis por outros objetos, dos detalhes internos da implementação daquele objeto, que ficam ocultos dos demais objetos. Ou seja, isola a parte externa da parte interna, facilitando o acesso e o uso do objeto. Por exemplo, não preciso saber como o cálculo da média do ALUNO é feita, só preciso saber que chamando o método CALCULAR

MEDIA ALUNO informando a MATRÍCULA do ALUNO, vou receber como retorno a MÉDIA do ALUNO.

- Herança: é o compartilhamento de atributos e operações entre classes com base em um relacionamento hierárquico. Permite que uma classe herde características de outra classe, permitindo aproveitar estas características que têm em comum. O conceito pode ser entendido também como classe mãe e classe filha, onde informações em comum são compartilhadas ou herdadas, e cada classe em si possui suas próprias características específicas. Imagine uma classe MEIO DE TRANSPORTE que possui características próprias, essa é a classe mãe. Herdando características em comum, temos a classe filha AUTOMÓVEL e CAMINHÃO, que possuem características próprias, como PLACA, por exemplo. PLACA é própria do AUTOMÓVEL ou CAMINHÃO, não estando na classe MEIO DE TRANSPORTE.



Neste exemplo, temos um sistema de uma escola onde as três classes modeladas foram ALUNO, TURMA onde o aluno está matriculado e PROFESSOR, que é o responsável pela turma.

É importante ressaltar que, como dito anteriormente, o objeto ou classe é uma representação do mundo real, relacionado com o cenário que se está modelando, neste caso, uma escola.

Continuando o entendimento do nosso exemplo de diagrama de classe. O ALUNO está matriculado em uma TURMA, e podemos dizer até que um aluno só possui uma única turma. A TURMA é formada por vários alunos. Um PROFESSOR ministra aulas em uma TURMA. O PROFESSOR, dependendo da sua disciplina, pode ministrar aulas em mais de uma TURMA. E uma TURMA pode ter mais de um PROFESSOR relacionado.

Todos estes relacionamentos são essenciais para entendermos como é a dinâmica da escola e, consequentemente, como o software deverá funcionar para atender a esta realidade.

Portanto, fica claro que dependendo da organização de cada escola, o software deve ser modelado e construído de acordo. Por isso a importância de levantar e entender bem os requisitos, e, após isso, modelar as classes de forma que implementem da melhor forma possível a necessidade do cliente.

Dessa forma, o Diagrama de Classes modela o entendimento sobre quais são as classes relacionadas com o escopo do software, assim como o relacionamento entre elas, mostrando como as informações se encaixam e se comunicam entre si para que o software faça tudo o que precisa ser feito para atender às necessidades do cliente.

Agora que entendemos o objetivo de um Diagrama de Classes, vamos entender um pouco mais como ele é construído.

3.1 Elementos de um diagrama de classe

Um Diagrama de Classe é formado por vários elementos que possuem objetivos próprios e, juntos, mostram as informações pertencentes a cada classe, além das ações sob responsabilidade de cada classe. Os elementos são:

- Classes: são os objetos do mundo real relacionados com o escopo do software a ser construído.
- Relacionamentos: é como as classes trocam informações entre si. Os relacionamentos podem ser de mais de um tipo, o que veremos a seguir.
- Métodos relacionados a cada classe: os métodos são as ações que estão sob responsabilidade de cada classe, ou seja, são as ações que devem ser executadas por cada classe para implementar o escopo pertencente a cada classe, por exemplo, o “Matricular Aluno” que discutimos anteriormente.

Pensando sobre este conceito de ações sob responsabilidade de cada classe, é importante perceber que uma classe de ALUNO não pode ter ações relacionadas com PROFESSOR, por exemplo. Quando as ações são organizadas corretamente na sua classe de origem, fica mais fácil entender o funcionamento do software.

Vamos a um exemplo deste conceito que acabamos de discutir. Imagine que uma das ações possíveis em um software de escola seja “Matricular Aluno”. Esta ação está relacionada a vincular um ALUNO novo a uma TURMA. Portanto, o ideal é esta ação ficar na classe ALUNO, e a classe ALUNO ter um relacionamento com a classe TURMA, para informar a qual TURMA o ALUNO pertence. Não faz nenhum sentido a ação de “Matricular Aluno” estar, por exemplo, na classe PROFESSOR.

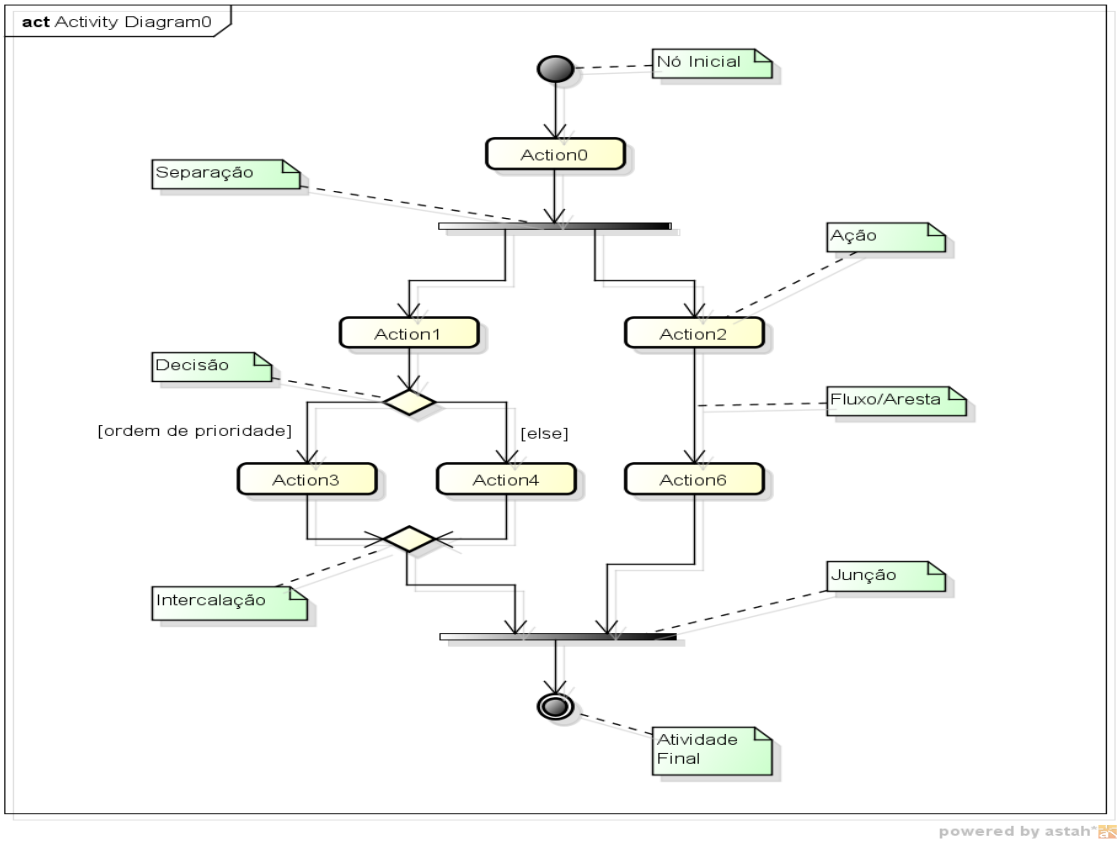
Dessa forma, o raciocínio utilizado para modelar um Diagrama de Classe é entender, logicamente, o funcionamento dos requisitos, pensando nas classes que fazem parte do software.

TEMA 4 – DIAGRAMA DE ATIVIDADE E A MODELAGEM DE FLUXO DE TRABALHOS EM PROCESSOS DE NEGÓCIOS

O Diagrama de Atividades é o diagrama da UML que corresponde ao diagrama BPM de processo de negócio estudado em etapas anteriores. Ele mostra as atividades e as mudanças de uma atividade para outra com os eventos ocorridos em alguma parte do sistema e ajuda a compreender o fluxo de controle de uma atividade para outra. Ou seja, este diagrama tem como objetivo descrever a lógica de uma funcionalidade, de um processo de negócio e mesmo de um fluxo de trabalho.

Os Diagramas de Atividades mostram as atividades que compõem um processo do sistema e seu fluxo de controle. Para modelar o Diagrama de Atividades é preciso decompor um processo em suas atividades, compreendendo quais atividades são sequenciais, quais são concorrentes e quais são executadas paralelamente em fluxo de negócio. Ou seja, é possível compreender que o Diagrama de Atividades é a forma da UML representar os processos automatizados por um software. Conforme falado anteriormente, o diagrama de atividades possui uma notação diferente, porém, possui o mesmo objetivo da modelagem BPM.

Visualmente, um Diagrama de Atividades possui a seguinte representação:



4.1 Elementos que compõem um diagrama de atividades

O Diagrama de Atividades, por representar uma sequência de ações, possui os seguintes elementos: estados iniciais e finais, atividades e transições, decisões, bifurcação e união, e raias.

- Estado inicial e final: todos os diagramas de atividades possuem pelo menos um estado inicial e pelo menos um estado final, mas pode haver vários estados, dependendo do fluxo de processos que se está modelando. O estado inicial indica o início do processo e o estado final indica o fim do processo e possuem as seguintes representações:

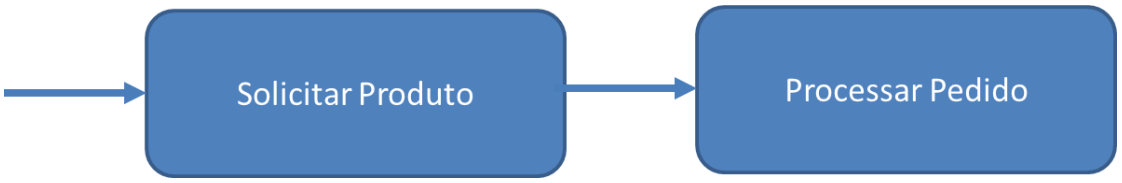
Estado Inicial (Iniciação)

Estado Final (Conclusão)

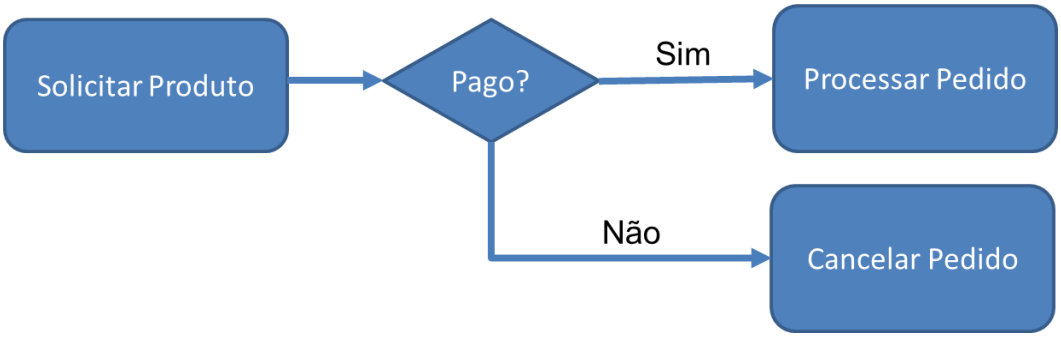
- Atividades: são as ações que devem ser executadas. Quando uma atividade é finalizada, a execução do fluxo do processo é transferida para

a próxima atividade. Essa transferência é chamada de transição. As atividades são representadas por retângulos com bordas arredondadas, no Diagrama de Atividades.

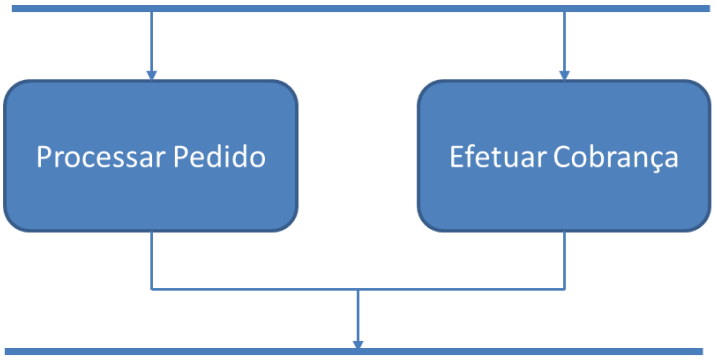
- Transição: é o caminho a ser seguido ao longo de todo o fluxo do processo, até a sua conclusão do processo. A transição é representada, no Diagrama de Atividades, por setas contínuas que representam a mudança no fluxo de trabalho de uma atividade para outra ao longo do processo que está sendo modelado.



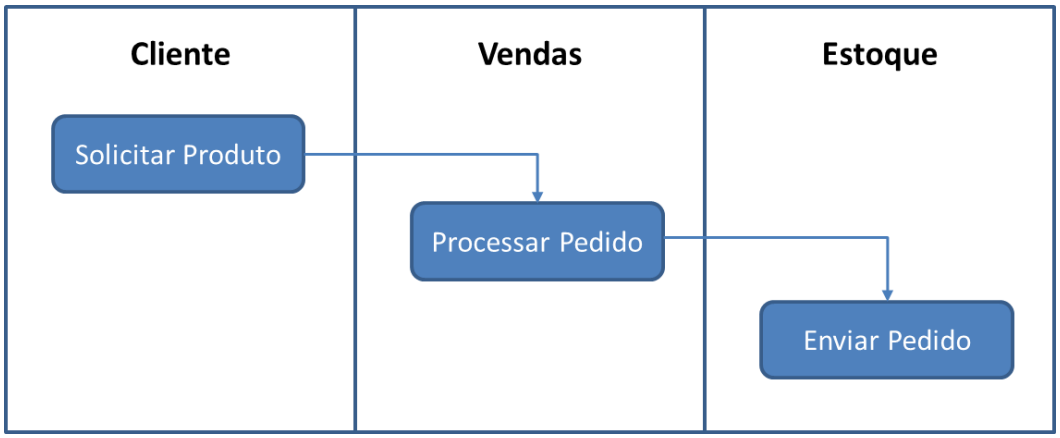
- Decisões: têm como objetivo controlar os desvios ao longo do fluxo do processo de negócio. Os desvios são descritos através de expressões lógicas e são representados, no Diagrama de Atividades, por um losango.



- Bifurcação e união: são a junção ou a separação de atividades executadas ao longo de um fluxo de negócio. A bifurcação é a divisão do fluxo de controle, enquanto a união é a sincronização ou junção das atividades realizadas paralelamente, em um único fluxo. Tanto a bifurcação quanto a união são representadas, no Diagrama de Atividades, com uma barra sólida.



- Raias: as raias são uma forma de organização lógica das atividades. Essa organização pode estar associada a objetos, a usuários ou atores ou a outra organização lógica que agregue valor para compreender o fluxo das atividades em um Diagrama de Atividades.



Com estes elementos, é possível verificar que o Diagrama de Atividades mostra, de maneira visual, como o fluxo de ações em um processo de negócio ocorre, mostrando as decisões que são tomadas ao longo do caminho entre o estado inicial e o estado final do processo.

Como podemos ver, o diagrama de atividade é uma opção para a modelagem de processos de negócios, pois fornece uma representação clara e detalhada dos fluxos de trabalho. Ele facilita a análise, documentação e otimização dos processos, além de apoiar a implementação de sistemas alinhados com as necessidades de negócio.

Agora você pode escolher entre usar o BPM ou o Diagrama de Atividades nos seus projetos, analise com qual modelo você melhor se identifica.

TEMA 5 – DIAGRAMA DE SEQUÊNCIA E A MODELAGEM DE INTERAÇÕES ENTRE OBJETOS NO CONTEXTO DE NEGÓCIOS

A modelagem de interações entre objetos no contexto de negócios refere-se à representação das trocas de informações e ações que ocorrem entre diferentes entidades (objetos) dentro de um sistema ou processo de negócio. Esses objetos podem ser pessoas, sistemas, subsistemas, componentes de software, ou qualquer outra entidade relevante que participe de um processo de negócio. A modelagem dessas interações é crucial para entender, documentar e otimizar como diferentes partes de um sistema trabalham juntas para atingir objetivos de negócio.

Para auxiliar no entendimento das interações de um software, vamos analisar mais um diagrama da UML e entender como ele nos ajuda no planejamento técnico de um projeto de software. Estamos falando agora sobre o diagrama de sequência que é um diagrama que ilustra a sequência das mensagens trocadas entre objetos em uma interação. Um diagrama de sequência consiste em um grupo de objetos relacionados por linhas de vida e apresenta, de maneira visual, as mensagens que eles trocam durante uma transação, desde o momento que a mensagem se inicia, passando por todos os componentes por onde ele passa, até o momento final, onde a mensagem chega no seu destino.

Dessa forma, um diagrama de sequência mostra a sequência de mensagens trocadas entre objetos, além disso, um diagrama de sequência também mostra as estruturas de controle entre objetos do software. Ou seja, um diagrama de sequência é representado através de duas dimensões: uma dimensão horizontal, que representa o conjunto de objetos e uma dimensão vertical que representa o tempo de vida de um objeto, podem representar também a criação e a destruição de objetos.

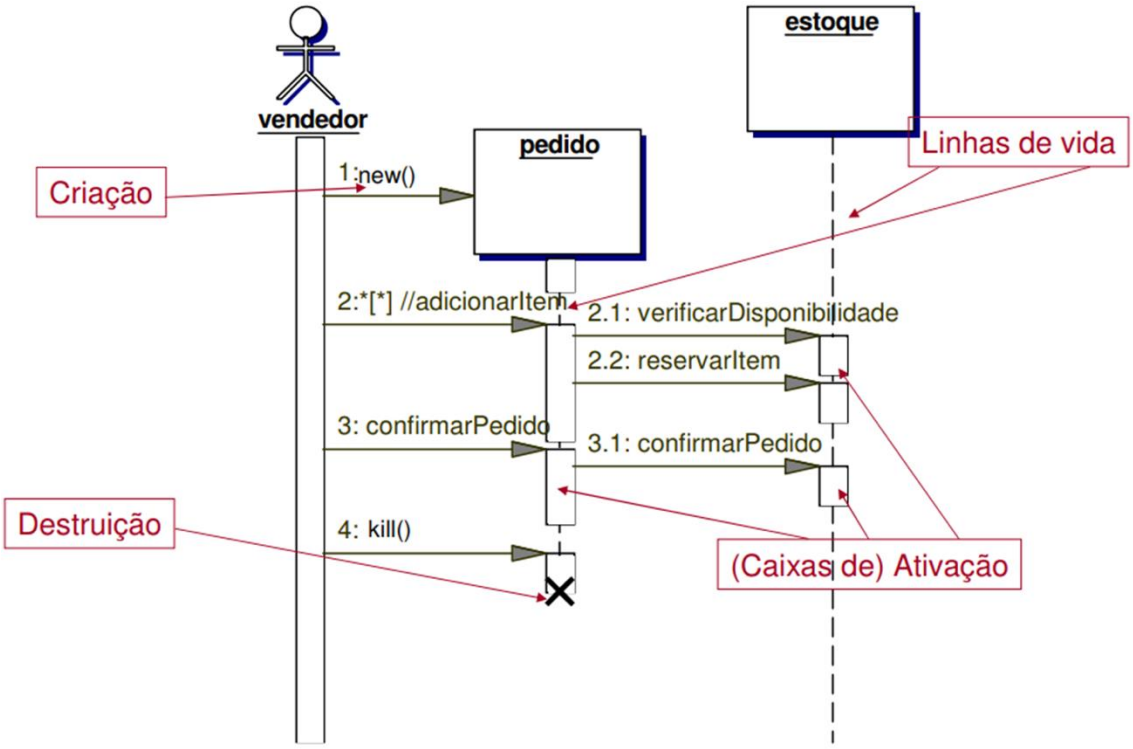
Por exemplo, linhas de vida em um diagrama de sequência para um cenário financeiro podem representar um cliente, um funcionário ou um gerente do banco, que são os objetos. A comunicação entre estes objetos é representada por mensagens transmitidas de um para o outro. Portanto, o diagrama de sequência mostra os objetos e as mensagens trocadas entre estes objetos.

5.1 Elementos do diagrama de sequência

Um diagrama de sequência, como todo diagrama, é composto por elementos que organizam o que o diagrama quer representar dentro do software.

- Ator: é o usuário que inicia a interação e a troca de mensagens. Um ator pode ser um usuário de sistema, uma funcionalidade ou um componente de um sistema externo.
- Linha de Vida: é uma instância ou uma ocorrência de um componente, onde chegam mensagens, e de onde partem mensagens. E o tempo de vida de um componente desde o momento em que este é chamado até quando ele é finalizado, concluindo seu objetivo.
- Fragmento: é onde tratamos as estruturas condicionais que fazem parte do fluxo de vida da mensagem em um objeto (os if/else, for/while e qualquer tratamento de exceção). O fragmento é representado pelo retângulo que se encontra no meio do fluxo de mensagem.
- Mensagem: é a mensagem “de fato” que trafega pela linha de vida. A mensagem é representada por uma seta, que segue a direção do fluxo das interações.

Bom, uma vez entendido os elementos do diagrama de sequência, vamos analisar um exemplo de diagrama. A escrita bem-feita, completa e clara de um caso de uso é fundamental para garantir uma boa comunicação entre as áreas de negócio e equipe técnica, e minimiza ruídos no entendimento do funcionamento dos requisitos, evitando problemas de requisitos no funcionamento do software.

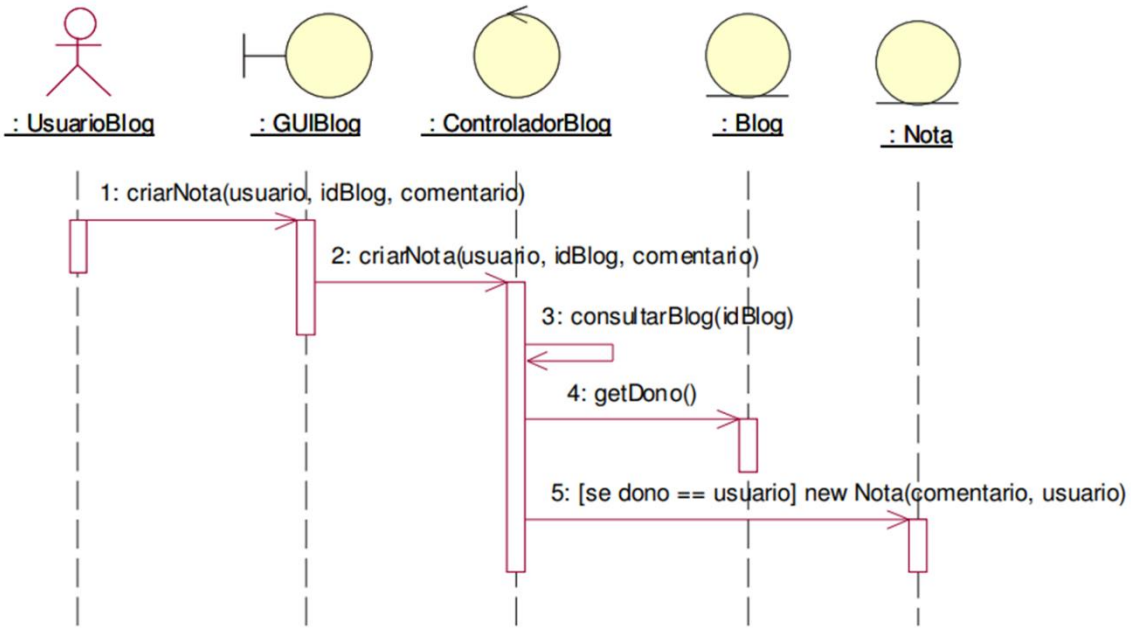


Neste exemplo podemos identificar a criação e a destruição do objeto PEDIDO, além de mostrar caixas de ativação, que indicam a chamada de um método do próprio objeto. Veja que o diagrama de sequência, assim como o diagrama de classe, estudado anteriormente, também utiliza o conceito de objetos e métodos.

O ator Vendedor é quem inicia a sequência de trocas de mensagens, mostrando fisicamente os objetos envolvidos, neste caso, PEDIDO e ESTOQUE e os métodos acionados para processar a mensagem recebida do ator.

As linhas da vida de cada objeto são representadas pela linha pontilhada, e mostram o ponto onde o objeto foi criado até o ponto quando ele é destruído, mostrando o final da instância do objeto utilizado na sequência do cenário modelado.

Vamos analisar outro exemplo, explorando agora os diversos tipos de objetos que podem ser inseridos em um Diagrama de Sequência de forma a mostrar o fluxo completo de uma mensagem trocada ao longo do processamento de uma funcionalidade. Lembre-se de que um Diagrama de Sequência representa uma funcionalidade descrita em um caso de uso.



No exemplo acima, observamos que o ator UsuarioBlog inicia o fluxo para criar uma nota em um blog (este é o caso de uso). Como objeto, temos a interface visual, :GUIBlog, onde o ator informa a nota que deseja inserir no blog. Os objetos :Blog e :Nota representam classes e entidades onde a nota em si será processada, tratada e armazenada.

Portanto, é possível perceber como a mensagem (neste exemplo, a nota de um blog) entra no sistema, é processada e armazenada, ou seja, o início e o fim do fluxo do caso de uso, mostrando fisicamente os objetos envolvidos na sequência.

Dessa forma, o Diagrama de Sequência utiliza a descrição do caso de uso como forma de compreender o funcionamento da funcionalidade que está sendo modelada, e utiliza o Diagrama de Classes para relacionar as classes ou objetos envolvidos na funcionalidade.

Este entendimento reforça o conceito de que, na análise e no projeto de software, os modelos criados vão mostrando e esclarecendo partes importantes do software, e o resultado de um diagrama ajuda a modelar e a compreender o próximo diagrama modelado, até que todos os aspectos do software estejam documentados e compreendidos.

É importante lembrar que não devemos gerar documentação desnecessária; a documentação deve agregar valor ao entendimento do

funcionamento do software como um todo. Documentação excessiva e sem propósito é um desperdício, o que vai contra os princípios ágeis.

Resumindo o que vimos neste tópico, esse diagrama é um tipo de diagrama de interação que ilustra como objetos interagem em um determinado cenário de execução, mostrando a sequência de mensagens trocadas entre eles ao longo do tempo.

FINALIZANDO

Chegamos ao final desta etapa. Espero que os conceitos discutidos tenham ficado claros. O mais importante é entender o objetivo e como deve ser representado um fluxo de processos de negócio utilizando a ferramenta BPM com sua notação BPMN.

Perceberam como a UML é rica para compreender o funcionamento e representar as soluções planejadas para a construção de um software? Aproveitem esse conhecimento para tornar seus projetos mais assertivos em relação às necessidades dos clientes e à melhor solução técnica para o projeto que estiverem construindo.

Espero que os exemplos práticos dos diagramas apresentados tenham ajudado a compreender o objetivo de cada um e como é possível modelar um software de forma visual.

Posteriormente, vamos continuar nos aprofundando em outros assuntos relacionados com a modelagem de processos de negócios para o desenvolvimento de um software.

REFERÊNCIAS

BECKER, J.; ROSEMANN, M.; VON UTHMANN, C. Guidelines of business process modeling. In: VAN DER AALST, W.; DESEL, J.; OBERWEIS, A. (Ed.). **Business Process Management**. New York: Springer Berlin Heidelberg, 2000. p. 30-49.

COHN, M. **User Stories Applied for Agile Software Development**. Assison-Wesley Professional. 2004.

GANE, C.; SARSON, T. **Análise Estruturada de Sistemas**. LTC, 1983.

LAPLANTE, P. A. **Requirements engineering for software and systems**. Boca Raton: CRC Press, 2013.

MALL, R. **Fundamentals of software engineering**. New Delhi: PHI Learning, 2014.

OGUNNAIKE, B. A.; RAY, W. H. **Process dynamics, modeling and control**. Oxford: Oxford University Press, 1994.

PAIM, R. et al. **Gestão de processos**: pensar, agir e aprender. Porto Alegre: Bookman. 2009. 328 p.

PIZZA, W. R. **A metodologia Business Process Management (BPM) e sua importância para as organizações**. Monografia (Curso de Tecnologia em Processamento de Dados) – Faculdade de Tecnologia de São Paulo, São Paulo, 2012.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**: uma abordagem profissional. 8. ed. Mc Graw Hill Education, 2016.

ROSEMANN, M.; BROCKE, J. V.; HONORATO, B. **Manual de BPM**: gestão de processos de negócio. Porto Alegre: Bookman, 2013.

SILVER, B.; RICHARD, B. **BPMN method and style**. Aptos: Cody-Cassidy Press, 2009. v. 2.

WIEGERS, K.; BEATTY, J. **Software requirements**. 3. ed. London: Pearson Education, 2013.